## Table of Contents

Malformed Parts: Causes, Detection, and Repair (2026-11-01)
TL;DR
1. Method
1.1 Three detectors, because no single one is enough
1.2 The deliberately broken corpus
1.3 The oracle, and a correction to how it was driven
2. The corpus's own malformation: twelve classes, three detection tiers
2.1 Tier 1 — element order (five classes, all caught by the schema)
2.2 Tier 2 — malformed values and wrong nesting (three classes)
2.3 Tier 3 — semantically wrong but structurally valid (four classes)
3. The editors' own malformations
3.1 office-open and Docen: one asymmetry, two symptoms
3.2 Folio: a model that normalises a move into a deletion
3.3 Fidus Writer: eleven schema violations, and a template's package
3.4 canvas-editor + plugin: a required attribute the writer forgot
3.5 docs.plus: the section properties are in the wrong place
3.6 The four that fail in Word, and the one that passes every static check
3.7 Plate.js: no malformation in the output — and the only importer that reports one
4. Does any editor repair malformed input?
4.1 The Word verdict is explained by the defect table — and it inverts the ranking
4.2 The broken corpus kills the editors that were faithful — and spares the ones that discard
4.3 What each defect class does to Plate.js — the per-defect detection matrix
4.4 What "repair" would even have looked like
5. The causes, in seven shapes
6. A detector catalogue
7. Corrections to the first report
Reproducing this

# Malformed Parts: Causes, Detection, and Repair (2026-11-01)

**A companion to *Open-Source DOCX Editors, Re-verified Against Code and Measured by Round-Trip (2026-11-01)*** and to *What Is Serialized, What Is Pass-Through, and What Is Actually Rendered (2026-10-30)*.

The first report found that the corpus it used — a hand-built 35-part adversarial DOCX — **Microsoft Word 16.115 would not open at all**, and located twelve defect classes behind that. This report closes the loop: what malformed parts look like, what causes them, and — the question the first report could not answer — **does any editor repair a malformed input, or does it preserve the damage?**

---

## TL;DR

- **Not one editor repairs anything.** Handed a corpus carrying all twelve defect classes, **every** editor either reproduced the damage or dropped the part that carried it. None diagnosed it, none fixed it — and, until **Plate.js** was added, not one said anything about it. **Docxodus, the thirteenth participant, does not change the finding: it preserves all ten package-visible defect classes, and Microsoft Word refuses its output.**
- **Docxodus is the only participant that ships a repair funnel, and it repairs a different set of defects.** Its `PackageOpenRepair` stage exists to unblock *"Package defects Word opens but the Open XML SDK rejects"* — a relationship to a part with **no content type** or **no ZIP entry**, a part reached by a known relationship but **typed only as generic XML**, and **two relationships sharing an Id**. **None of those is one of this report's twelve defect classes.** Having a repair stage and repairing a given malformation are different properties, and this is the cleanest available demonstration of the difference.
- **Plate.js is the exception, and it is a real one.** It is the only editor in the study that **detects** malformation and **refuses** the document: handed the broken corpus it fails with `decode-failed: DOCX content could not be decoded by the installed editor schema.`, and it produces **no output file at all** — there is nothing to corrupt. It also refuses the *clean* corpus under its own default `lossPolicy: 'reject'`, with 18 typed errors naming each construct it cannot represent. Plate does not repair malformation either. It **declines to process it**, which is the only honest of the three available behaviours (repair, degrade, refuse).
- **And Plate detects exactly one of the twelve defect classes.** Each defect was isolated into its own single-defect corpus and offered to Plate's importer: it **accepts eleven and refuses one** — the block-level `m:oMathPara`. The eleven it accepts include every class that makes Word refuse the file: the five element-order classes, the malformed `ST_LongHexNumber`, the nested `mc:Fallback`, the broken OLE payload, `w:delText` inside `w:moveFrom`, and `w:dirty="true"`. Plate is a detector, not a validator, and it detects the construct its own schema cannot model.
- **Four of the fourteen editor outputs cannot be opened by Microsoft Word**, and only one of those four (Fidus Writer) fails for a reason a schema validator could have caught; a package-integrity check catches two more, and Folio passes both:

| Editor | Real XSD violations | Package integrity | Word 16.115 |
|---|---|---|---|
| SuperDoc | 0 | clean | **opened** |
| GenOffice | 0 | clean | **opened** |
| eigenpal | 0 | clean | **opened** |
| WordCraft | 0 | clean | **opened** |
| **Docxodus** | **0** | **clean** | **opened** |
| canvas-editor + plugin | 1 | clean | **opened** |
| docs.plus | 1 | clean | **opened** |
| **Plate.js** | **0** | **clean** | **opened** |
| LibreOffice (baseline) | 0 | clean | **opened** |
| Folio | 0 | clean | **REFUSED** |
| office-open | 0 | **5 dangling relationship ids** | **REFUSED** |
| Docen | 0 | **5 dangling relationship ids** | **REFUSED** |
| Fidus Writer | 11 | clean | **REFUSED** |

- **Two refusal causes are now proven causally, not just correlated.** Both were isolated to a single attribute or a single identifier, and in both cases *writing that one thing back correctly makes Word open the file*:

| Editor | Cause | One-line repair | Result |
|---|---|---|---|
| Folio | the importer rewrites a move source's `w:t` as `w:delText` | `w:delText` → `w:t` inside `w:moveFrom` | **REFUSED → opened** |

| Editor | Cause | One-line repair | Result |
|---|---|---|---|
| office-open / Docen | the body is rewritten with numeric relationship ids while the `.rels` keeps the source's descriptive ones | rewrite 5 ids to the declared names | **REFUSED → opened** |

- **The office-open / Docen defect is a single asymmetry with two symptoms.** One id-resolution function accepts only numeric relationship ids and the other does not. The dangling reference is symptom one; the "unbound prefix" malformed XML on re-import is symptom two, and it is why those two projects are the only non-idempotent ones in the study (second-pass scores 32.0% and 30.5%).
- **The broken corpus kills the editors that were faithful to it, and spares the ones that discard the content — and d12 is no longer one of the defects that kill.** SuperDoc, GenOffice, eigenpal, Docxodus and Apryse WebViewer open the clean corpus and refuse the broken one. WordCraft, canvas-editor + plugin, docs.plus and LibreOffice (baseline) open both. canvas, docs.plus and LibreOffice opened it because they dropped the content that carried the defects. WordCraft kept `w:dirty="true"` (d12), which this release finds is not fatal on its own (§2). **Fidelity and robustness are anti-correlated here**, and the first report's ranking does not survive a malformed input unchanged.
- **Plate.js is the twelfth editor, and it breaks the pattern in the most useful way.** It cannot open the broken corpus either — but it fails **before writing anything**, so it is the only editor in the study that refuses a malformed file without leaving a damaged one behind. Its clean-corpus output, by contrast, is schema-clean, package-clean and **opens in Word**. **Docxodus is the thirteenth, and it does not break the pattern: it is faithful, it preserves the damage, and Word refuses its broken-corpus output — exactly as the defect table predicts.**

---

# 1. Method

## 1.1 Three detectors, because no single one is enough

| Detector | Finds | Blind to |
|---|---|---|
| **ECMA-376 Transitional XSD** (`build/validate_ooxml.py`) | element order, missing required attributes, wrong value types | anything in a namespace the schema set does not model; anything that is semantically wrong but structurally fine |
| **Package-integrity auditor** (`build/malformation_audit.py`) | not-well-formed XML, dangling `r:id`, relationship targets that do not exist, parts with no content type, stale overrides, unbalanced fields, unpaired moves | anything that is a valid package with wrong content |
| **Microsoft Word itself** | everything, in the only verdict that matters | it gives no diagnosis — the refusal is a modal dialog and the app's own AppleScript surface cannot read it |

A defect class that all three miss is a defect class that only a human reading the XML will catch. The first report found exactly such a class three times.

## 1.2 The deliberately broken corpus

`build/make_broken_fixture.py` rebuilds the corrected corpus with all twelve defect classes re-introduced — the ordering errors, the malformed `ST_LongHexNumber`, the misplaced `dgm:style`, the nested `mc:Fallback`, the fake OLE payload, the `w:delText` inside `w:moveFrom`, the block-level `m:oMathPara` and the `w:dirty="true"` flag.

The rebuilt file is a faithful reproduction of the original defect set: `build/validate_ooxml.py` reports ten real schema violations against it (the ordering and value classes), and Word refuses it.

**Microsoft Word 16.115 refuses `fixture/complex_broken.docx` outright**, exactly as it refused the original.

## 1.3 The oracle, and a correction to how it was driven

Word is asked one question per file: does it open it? The harness (`build/word_open.py`) restarts Word, waits for the AutoRecovery restore to land, clears the document set, opens the file and polls for a document whose **name matches**.

Two failure modes of that harness had to be fixed before its verdicts could be trusted, and both are worth recording because either one produces a plausible, wrong answer:

- **Name collision.** Every editor's output is called `complex.docx`. A poll that matches on the base name will happily report "opened" because of a *different* file that happens to still be open. Every file is therefore staged under a unique name before the test.
- **The restore steals the open.** A hard kill makes Word restore its AutoRecovery set on the next launch — including the user's own documents. That restore lands *after* a short sleep, so the target file is never opened and the poll sees somebody else's document. The harness now waits for the restore, clears it, and confirms the document count is zero before opening the target.

A third one surfaced later, and it is the reason every verdict in this report is a **double** measurement:

- **A busy Word queues the open instead of executing it.** Closing a document leaves Word processing the close for a moment afterwards; an Apple Event that arrives in that window is queued rather than handled, the file is never opened, and the poll sees an empty document list and reports **REFUSED** for a file Word would have accepted. This produced one false refusal — of this report's own `.docx` — which vanished on a second, isolated run. The harness now waits for the document count to reach zero and for the app to settle before sending the open, and **every refusal is retried once against a freshly restarted Word**. A verdict is only reported as REFUSED when both attempts agree, which is why the four refusals in §3.6 are solid and why the clean-corpus matrix in §4.2 was re-run in full under the hardened harness.

With the harness fixed, the corpus opens, and the verdicts in this report are reproducible.

---

## 2. The corpus's own malformation: twelve classes, three detection tiers

The first report established that Word refused the corpus. Re-reading that with the three detectors in hand, the twelve classes fall into three tiers, and the tier determines which detector finds them.

### 2.1 Tier 1 — element order (five classes, all caught by the schema)

`CT_Settings`, `CT_TcPr`, `CT_TrPr`, `CT_PPrBase` and `CT_SectPr` are **sequences**, not choices. Putting a child in the wrong position is invalid even though every element is individually legal.

| # | Defect | Where | Sequence broken |
|---|---|---|---|
| 1 | `w:documentProtection` after `w:defaultTabStop`/`w:evenAndOddHeaders` | `word/settings.xml` | `CT_Settings` |
| 2 | `w:tcW` after `w:gridSpan`/`w:vMerge` | every `w:tcPr` | `CT_TcPr` |
| 3 | `w:cantSplit` after `w:tblHeader` | `w:trPr` | `CT_TrPr` |
| 4 | `w:spacing` after `w:jc` inside `w:pPrChange/w:pPr` | `w:pPrChange` | `CT_PPrBase` |
| 5 | `w:pgNumType` after `w:cols` | the landscape `w:sectPr` | `CT_SectPr` |

### 2.2 Tier 2 — malformed values and wrong nesting (three classes)

| # | Defect | Rule broken |
|---|---|---|
| 6 | `w16cid:durableId="1234567890"` | `ST_LongHexNumber` is four bytes — exactly eight hex digits |
| 7 | `dgm:style` as a direct child of `dgm:styleDef` | a style definition is a choice of *labelled* styles; the label is the container |
| 8 | `mc:Fallback` nesting a `w:r` inside the outer `w:r` | the fallback holds a `w:pict` directly, not a run |

### 2.3 Tier 3 — semantically wrong but structurally valid (four classes)

These are the interesting ones, because **no schema validator reports any of them**. They are also the ones that cost the most to find, because the only way to detect them is to ask a consumer that is stricter than the schema.

| # | Defect | Why it is fatal |
|---|---|---|
| 9 | the OLE object points at 64 bytes of filler | Word embeds a Word document as the raw `.docx` bytes behind `o:OLEObject ProgID="Word.Document.12"`; it tries to open the payload and fails |
| 10 | `w:delText` inside `w:moveFrom` | `w:delText` marks text to be displayed "when the run is part of a deletion". A move source is not a deletion. Word reads the move as a second, unmatched deletion |
| 11 | `m:oMathPara` as a direct child of `w:body` | the transitional schema permits a block-level equation through `EG_ContentBlockContent` → `EG_RunLevelElts` → `m:EG_OMathMathElements`; Word requires the equation inside a `w:p`, the way Word itself writes it |
| 12 | `w:dirty="true"` on a `w:fldChar` | `w:dirty` is an ordinary `ST_OnOff` attribute, and Word itself writes it on an out-of-date table of contents. The first report found it fatal to the whole document; this release does not (below) |

**The first report's table for #12**, then the least intuitive result in the study (the attribute variants were not re-run in this release):

| `w:fldChar` attribute | Word, first report |
|---|---|
| `w:dirty="true"` | **refused** |
| `w:dirty="1"` | **refused** |
| `w:dirty="false"` | opened |
| `w:fldLock="false"` | opened |
| attribute absent | opened |

It does not depend on the field type (a `PAGE` field refuses identically), on `w:documentProtection`, or on the rest of the package. It is not specific to the corpus: **pandoc writes `w:dirty="true"` on the TOC field of every document it produces**, so the first build of the first report's own `.docx` was refused by Word for exactly this reason, and `build/fix_dirty.py` now rewrites it to `"false"` in every generated file.

**In this release the rule did not hold.** The single-defect corpora (`build/make_single_defect.py`) were opened again through the audited driver, twice: the clean corpus opened, d10 alone did not open, d11 alone did not open, and **d12 alone opened**, with the same text as the clean corpus (`out/word_isolation.json`). It opened after Word asked *"This document contains fields that may refer to other files. Do you want to update the fields in this document?"* — a question, not an offer to repair, but a warning all the same, so a file carrying d12 opens and is still not Word valid. WordCraft's broken-corpus output, which keeps d12 and drops d10 and d11, opened too (§4.1). The first report used the same Word version through a hand-written driver that kept no record of what Word showed, and a driver that reads that question as a refusal produces the first report's table exactly. That is the likely explanation, not a proven one; the driver used now names the document it saves, declines Word's repair prompt, and writes a manifest. `build/fix_dirty.py` still rewrites `w:dirty` to `"false"` in every generated file, which is harmless either way.

## 3. The editors' own malformations

### 3.1 office-open and Docen: one asymmetry, two symptoms

This is the most interesting defect in the study, because it is a single coding inconsistency that produces a dangling package *and* malformed XML, and because the two symptoms appear in different passes.

**The asymmetry.** office-open resolves the same relationship twice, with two different rules:

- `packages/docx/src/compile/document.ts:39-61`, `sourceRidFor()`, extracts the numeric part of the source relationship id with `/^rId(\d+)$/`. The corpus uses **descriptive** ids — `rIdChart`, `rIdDgmData`, `rIdDgmLayout`, `rIdDgmQuickStyle`, `rIdDgmColors` — so the match fails, the function returns `undefined`, and the body falls back to `formatId(chartOffset, i, "rId")`, producing `rId17`.
- `packages/docx/src/compile/document.ts:78-100`, `sourceRelationshipFor()`, has **no** numeric restriction. It finds the relationship and returns its real id, `rIdChart`.

So the regenerated body says `rId17` while the relationship file still says `rIdChart`. The body references five relationship ids that the package never declares:

```
<w:drawing><wp:inline>…<a:graphic><a:graphicData uri="…/chart">
  <c:chart r:id="rId17"/>
```

and, for the SmartArt diagram:

```
<dgm:relIds r:dm="rId18" r:lo="rId19" r:qs="rId20" r:cs="rId21"/>
```

`build/malformation_audit.py` reports exactly five dangling references for office-open and five for Docen. **The chart and the SmartArt diagram are in the file and the body points at them, but nothing binds them to the package.** The first report's checker only asked whether the `c:chart` element was present, so it scored this 100% text retention while the chart was in fact unresolvable.

**Symptom two: the malformed XML on the second pass.** When office-open reads its own output back, `rId17` cannot be resolved, so the typed chart path is skipped and the "unsupported drawing" fallback re-synthesizes an `<a:graphic>` element. The namespace registry at `packages/docx/src/compile/body.ts:426-459` (`DOC_NAMESPACE_KEYS`) does not include `a`, so the re-synthesized element carries no `xmlns:a` binding — and the regenerated `<w:document>` root declares no `a` prefix either. The result is a file that is not well-formed XML:

```
unbound prefix: line 1, column 13606
```

The image's `<a:blip>` drawing keeps its binding because it takes a different path. That asymmetry — one drawing path that knows about the namespace registry and one that does not — is why only the chart and SmartArt break.

**Proof.** The repair is one substitution. Rewriting `rId17`→`rIdChart`, `rId18`→`rIdDgmData`, `rId19`→`rIdDgmLayout`, `rId20`→`rIdDgmQuickStyle`, `rId21`→`rIdDgmColors` in the body — changing nothing else — turns Word's verdict from **REFUSED** into **OPENED**.

**Consequence.** These two projects are the only non-idempotent ones in the study: their second-pass scores collapse to 32.0% and 30.5%, and their second-pass `word/document.xml` does not parse at all.

### 3.2 Folio: a model that normalises a move into a deletion

Folio's importer rewrites the move source's text:

```
input : <w:moveFrom …><w:r><w:t xml:space="preserve">moved away</w:t></w:r></w:moveFrom>
output: <w:moveFrom …><w:r><w:delText>moved away</w:delText></w:r></w:moveFrom>
```

That is defect #10 from §2.3, introduced by the editor itself. **Microsoft Word 16.115 refuses Folio's output.** Folio scores 98.5% on the first report's checker, keeps 34 of 35 parts byte-identical, and has no schema violation at all — and its output will not open.

**Proof.** Rewriting `w:delText` back to `w:t` in that one element — changing nothing else — turns Word's verdict from **REFUSED** into **OPENED**.

This is a satisfying result for the study: the defect class that the first report discovered *by hand* in its own corpus is the same defect class a mature editor produces automatically. Folio could not have avoided it by trying harder; the corpus's original markup was simply the markup a real editor emits.

### 3.3 Fidus Writer: eleven schema violations, and a template's package

Fidus's output is the least malformed in the "not well-formed" sense — it is well-formed XML with no dangling references — but it carries **eleven real schema violations**:

- `word/settings.xml`: `<w:footnotePr>` emitted **after** `<w:compat>`, breaking the `CT_Settings` sequence;
- `word/numbering.xml`: `<w:start>` elements emitted **without the required `w:val` attribute** (`ST_DecimalNumber` is required and the value is literally `undefined`), and `<w:abstractNum>` elements emitted out of sequence.

The cause is architectural. Fidus's exporter takes a **template URL** (`src/exporter/docx/index.ts:34`) and fills that template's placeholders (`src/exporter/docx/render.ts:746-834`, `:982-1021`; placeholder vocabulary in `src/exporter/template/tags.ts:18-101`). The `word/styles.xml`, `word/numbering.xml` and `word/settings.xml` in the output are the **template's**, not the corpus's — so their defects are the template's defects, faithfully carried across.

### 3.4 canvas-editor + plugin: a required attribute the writer forgot

One real violation: `<w:shd>` emitted **without the required `w:val` attribute**. `w:shd` is `CT_Shd`, whose `w:val` is `use="required"`. The `docx` npm package's table-cell shading helper writes the fill colour but not the shading pattern.

The rest of canvas-editor's output is not malformed; it is simply a **different document**. The exporter never opens the source package at all — `jszip` is imported only by the importer — so there is no copy path to malform. Its output is the `docx` package's default part set: no `word/theme/theme1.xml`, no `word/webSettings.xml`, media re-emitted under random nanoid names (`docx@8.6.0`, `build/index.mjs:11191`, `:10332`), and the source's `word/media/image1.png` reappearing as `word/media/zsakc18onfp21xzadj5t0.png` with byte-identical content.

### 3.5 docs.plus: the section properties are in the wrong place

One real violation, and it is a good one:

```
<w:body>
  <w:sectPr>
    <w:pgSz w:w="12240" w:h="15840" w:orient="portrait"/>
    <w:pgMar …/>
  </w:sectPr>
  <w:p> …
```

In `CT_Body`, `w:sectPr` must be the **last** child — it describes the final section, and a section with no content after it is meaningless. docs.plus's pipeline is DOCX → HTML → Tiptap → HTML → DOCX, and **HTML has no section model**, so the converter emits a default `sectPr` — and puts it first.

That single misplacement is the visible symptom of the whole pipeline's limitation, and it explains the rendered comparison in the companion report: docs.plus renders the corpus in **3 pages instead of 6**, with 61 rendered lines that were never in the corpus, because the page borders, the landscape section and the multi-column section went through HTML and did not come back.

### 3.6 The four that fail in Word, and the one that passes every static check

| Editor | XSD | Package integrity | Word |
|---|---|---|---|
| Folio | clean | clean | **refused** |
| office-open | clean | **5 dangling ids** | **refused** |
| Docen | clean | **5 dangling ids** | **refused** |
| Fidus Writer | **11 violations** | clean | **refused** |

Folio is the sharpest illustration of the gap: **it is schema-clean, well-formed, has no dangling references, and does not open.** A validator run over it reports nothing at all.

### 3.7 Plate.js: no malformation in the output — and the only importer that reports one

Plate is the only editor in the study whose **output** has nothing wrong with it *and* whose **importer** complains. Its clean-corpus output is schema-clean against the 69 ECMA-376 transitional XSDs, package-clean (no dangling relationship ids, no missing targets, no stale overrides, no unbalanced fields, no unpaired moves), and it **opens in Microsoft Word 16.115**. That is not nothing: Fidus, canvas-editor and docs.plus all fail one of those checks, and Folio, office-open and Docen fail Word.

What is remarkable is the other side. Plate's importer does not merely fail on the malformed corpus — it **refuses**, before writing anything, with a named diagnostic:

> `decode-failed`: DOCX content could not be decoded by the installed editor schema.

And on the *clean* corpus, under its own default `lossPolicy: 'reject'` (`import/lib/importDocx.ts:2455`), the same refusal happens for a different and much more interesting reason: the corpus contains constructs Plate's model cannot represent, so the import produces **18 errors** across 38 typed diagnostics and fails. Under `lossPolicy: 'allow'` — which the harness uses so that an output exists to measure — the same 38 diagnostics are returned as warnings and the import proceeds.

So Plate has a three-state behaviour on a document it cannot fully represent: **refuse** (the default), **proceed with a full account of the loss** (`'allow'`), or **proceed silently** (which no editor here offers and none should).

It also has the only *provable* byte-exactness gate in the study. `retainSource: true` keeps the admitted package alive, and `internal/sourceEligibility.ts:453-582` admits it only if every part, relationship and inspected markup sits inside a **closed passive vocabulary** — exactly one `officeDocument` relationship to `word/document.xml` (`:531-542`), no external relationships except absolute hyperlinks (`:500-514`), raster parts matching their declared type's magic bytes (`:543-547`), every part reachable in the relationship graph (`:560-567`), and only `PAGE` recognised as a field instruction (`:128`). The corpus **fails** the last-but-one test, and Plate says so precisely:

> `source-unavailable`: The DOCX source cannot be retained because `word/_rels/document.xml.rels` declares an unrecognized relationship (chart); export will generate the document from editor content.

This is the closest any editor in this study comes to *proving* a round-trip guarantee, and it is notable that the proof is **negative** here: Plate declines the guarantee rather than asserting it.

---

## 4. Does any editor repair malformed input?

This is the question the first report could not answer, and it is the one that matters for a toolchain.

`fixture/complex_broken.docx` was pushed through all fourteen participants. Each output was then scanned for the ten defect classes that are visible in a package (the OLE and nesting classes are covered by the earlier table), and separately opened in Word.

YES = the defect class is still present in the editor's output · no = the class is gone · DROP = the whole part carrying it was dropped.

| Editor | d1 order (settings) | d2 order (tcPr) | d3 order (trPr) | d4 order (pPrChange) | d5 order (sectPr) | d6 durableId | d9 OLE payload | d10 delText in moveFrom | d11 block math | d12 dirty=true | Word 16.115 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| SuperDoc | YES | YES | YES | YES | YES | YES | YES | YES | YES | YES | **REFUSED** |
| Folio | YES | YES | YES | no | no | YES | YES | YES | YES | YES | **REFUSED** |
| GenOffice | YES | YES | YES | YES | YES | YES | YES | YES | YES | YES | **REFUSED** |
| eigenpal | YES | YES | YES | YES | YES | YES | YES | YES | YES | YES | **REFUSED** |
| Docxodus | YES | YES | YES | YES | YES | YES | YES | YES | YES | YES | **REFUSED** |
| office-open | no | no | no | no | no | no | YES | YES | YES | no | **REFUSED** |
| Docen | no | no | no | no | no | no | YES | YES | YES | no | **REFUSED** |
| WordCraft | YES | no | no | no | no | no | YES | no | no | YES | **OPENED** |
| Fidus Writer | no | no | no | no | no | no | DROP | no | no | no | **REFUSED** |
| Plate.js | — | — | — | — | — | — | — | — | — | — | no output |
| canvas-editor + plugin | no | no | no | no | no | no | DROP | no | no | no | **OPENED** |
| docs.plus | no | no | no | no | no | no | DROP | no | no | no | **OPENED** |
| Apryse WebViewer | no | no | YES | no | no | no | YES | YES | no | YES | **REFUSED** |
| LibreOffice (baseline) | no | no | YES | no | no | no | DROP | no | no | no | **OPENED** |

**Not one editor repaired a defect.**

- The **pass-through editors** — SuperDoc, GenOffice, eigenpal, and to a degree Folio — reproduce the damage byte-for-byte, because they never parsed it. That is the honest consequence of pass-through, and it is invisible in every score.
- The **generators** — office-open and Docen — rebuild the document and therefore *lose* the ordering defects by accident, because their writers emit elements in schema order. But they **reproduce the semantic damage**: a `w:delText` inside a `w:moveFrom` and a block-level `m:oMathPara` come out of them exactly as they went in. They are faithful and unvalidating. A generator that is faithful will happily emit a file Word refuses.
- **WordCraft** is the only editor that removed the move-source defect and the block-math defect — because it re-renders the move as its own model and never models `m:oMathPara` as a block. It removed them by *dropping the features*, not by repairing them.
- **Fidus Writer, canvas-editor and docs.plus** show no across the board because they **dropped the parts that carried the defects**, not because they understood them. Their DROP in the OLE column is the tell: the OLE object is gone because the whole payload was discarded.
- **Plate.js has no row at all, and that absence is the finding.** Its importer **refuses** the broken corpus — `decode-failed: DOCX content could not be decoded by the installed editor schema.` — so it writes **no output file at all**. There is nothing for a defect-survival scan to find. It is the only editor in the study that, on malformed input, produces **nothing** rather than producing something damaged.
- **LibreOffice** is the only one that repaired the `w:trPr` ordering on its own (`w:cantSplit`/`w:tblHeader`) while dropping everything else — because its writer emits rows in schema order.
- **Docxodus preserves all ten, and the mechanism is worth naming.** It is a pass-through writer for the twenty-five parts the Open XML SDK does not model and a re-serialiser for the ten it does — and `word/document.xml` and `word/settings.xml` are both in the second group. **Re-serialisation is faithful to the *tree*, not to the *schema*: the SDK prints the elements it read in the order it read them**, so an out-of-order `w:tcPr` comes back out of order. Its broken-corpus output therefore carries **ten real XSD violations** (7 × `w:tcW`, 1 × `w:spacing`, 1 × `w:pgNumType`, 1 × `w:documentProtection`) and is **package-clean** — the same shape of failure as the other four faithful editors, reached by a completely different route.

**The honest summary: an editor that does not validate will not repair. It will either preserve the damage, or lose the feature that contained it.**

### 4.1 The Word verdict is *explained* by the defect table — and it inverts the ranking

The last column of the table above is not an independent measurement; it falls out of the other columns. This is worth stating precisely, because the pattern is the single most useful result in this report.

The first report isolated three defect classes as fatal to Word on their own: **d10** (`w:delText` inside `w:moveFrom`), **d11** (block-level `m:oMathPara`) and **d12** (`w:dirty="true"` on a `w:fldChar`). In this release d10 alone did not open and d11 alone did not open, but d12 alone opened (§2). With d10 and d11 as the fatal classes, every editor's Word verdict is consistent with its own defect table, Fidus Writer apart:

| Editor | d10 | d11 | d12 | Word verdict | Of d10, d11, d12 in the output |
|---|---|---|---|---|---|
| SuperDoc | YES | YES | YES | REFUSED | all three |
| Folio | YES | YES | YES | REFUSED | all three |
| GenOffice | YES | YES | YES | REFUSED | all three |
| eigenpal | YES | YES | YES | REFUSED | all three |
| Docxodus | YES | YES | YES | REFUSED | all three |
| office-open | YES | YES | no | REFUSED | two |
| Docen | YES | YES | no | REFUSED | two |
| WordCraft | no | no | YES | OPENED | one (d12) |
| Fidus Writer | no | no | no | REFUSED | none |
| Plate.js | — | — | — | no output | the importer refused the corpus |
| canvas-editor + plugin | no | no | no | OPENED | none |
| docs.plus | no | no | no | OPENED | none |
| Apryse WebViewer | YES | no | YES | REFUSED | two |
| LibreOffice (baseline) | no | no | no | OPENED | none |

Three of these rows are individually decisive:

- **WordCraft separates d12 from the other two.** It is the only editor that removed d10 and d11 and kept d12. In v0.9.1 it was recorded as refused, which read as confirmation that `w:dirty="true"` alone is fatal. In this release the same output file opens, twice on each of two sets of outputs, and so does the d12-only corpus: nothing fatal survives in WordCraft's output, and Word opens it.
- **Fidus Writer's refusal is its own, not the corpus's.** Fidus dropped every defect and still could not open. It was *also* refused on the clean corpus (§3.3, eleven schema violations), so the broken corpus changed nothing for it. This is the one row where "the corpus killed it" is the wrong explanation, and it is the row that prevents the table from being read as a rule.
- **Plate.js's "refusal" is a third thing again.** It is neither a Word refusal nor an output failure: **Plate never wrote a file**. The import failed with `decode-failed`, so there is nothing for Word to judge. This is the only row in the table where the verdict column describes the *editor declining to act*, and it is the row that makes the other eleven legible — a refusal, a failure and a refusal-to-refuse are three different outcomes, and only one of them is safe.

### 4.2 The broken corpus kills the editors that were faithful — and spares the ones that discard

Putting the two Word matrices side by side is the clearest single statement of what pass-through costs.

| Editor | Clean corpus | Broken corpus | Verdict flipped by the corpus? |
|---|---|---|---|
| SuperDoc | OPENED | REFUSED | **yes** |
| Folio | REFUSED | REFUSED | no |
| GenOffice | OPENED | REFUSED | **yes** |
| eigenpal | OPENED | REFUSED | **yes** |

| Editor | Clean corpus | Broken corpus | Verdict flipped by the corpus? |
|---|---|---|---|
| Docxodus | OPENED | REFUSED | **yes** |
| office-open | REFUSED | REFUSED | no |
| Docen | REFUSED | REFUSED | no |
| WordCraft | OPENED | OPENED | no |
| Fidus Writer | REFUSED | REFUSED | no |
| Plate.js | OPENED | no output | **yes — the importer refused the corpus** |
| canvas-editor + plugin | OPENED | OPENED | no |
| docs.plus | OPENED | OPENED | no |
| Apryse WebViewer | OPENED | REFUSED | **yes** |
| LibreOffice (baseline) | OPENED | OPENED | no |

**Five editors that were faithful to the clean file die on the malformed one. Of the four that open both, three are the ones that throw the content away; the fourth, WordCraft, kept only d12, which Word now opens (§2).** Fidelity and robustness are anti-correlated here, and the reason is structural rather than accidental: an editor that preserves the document preserves the damage inside it, because the damage *is* the document as far as a pass-through writer is concerned. The editors that survive the malformed input are the ones that were already losing most of the document — they lose nothing more.

That is the uncomfortable trade a fidelity score hides completely. The first report's ranking puts SuperDoc first and canvas second-to-last. On malformed input the order **inverts**: canvas and docs.plus open, and SuperDoc does not.

**Plate.js is the one editor that is on the right side of this trade for the right reason.** Its clean-corpus output opens; its broken-corpus *import* fails. It is not that Plate is robust — it is that Plate **checks**. Its importer runs the document through the editor's own schema and, finding a construct it cannot model, **stops**. The result is that a malformed document produces no output at all rather than a plausible-looking wrong one.

### 4.3 What each defect class does to Plate.js — the per-defect detection matrix

The broken corpus carries all twelve defect classes at once, so a refusal there says only "something in here is bad". To find out **which** defects an editor can actually see, each defect class was isolated into its own single-defect corpus (`build/make_single_defect.py`) and offered to Plate's importer on its own.

| # | Defect class | Plate's verdict | Diagnostic |
|---|---|---|---|
| d1 | `w:documentProtection` after `w:evenAndOddHeaders` | **accepted** | — |
| d2 | `w:tcW` after `w:gridSpan`/`w:vMerge` | **accepted** | — |
| d3 | `w:cantSplit` after `w:tblHeader` | **accepted** | — |
| d4 | `w:spacing` after `w:jc` in `w:pPrChange` | **accepted** | — |
| d5 | `w:pgNumType` after `w:cols` in the landscape `w:sectPr` | **accepted** | — |
| d6 | `w16cid:durableId` with ten hex digits | **accepted** | — |
| d7 | `dgm:style` as a direct child of `dgm:styleDef` | **accepted** | — |
| d8 | `mc:Fallback` nesting a `w:r` inside the outer `w:r` | **accepted** | — |
| d9 | the OLE object pointing at 64 bytes of filler | **accepted** | — |
| d10 | `w:delText` inside `w:moveFrom` | **accepted** | — |

| # | Defect class | Plate's verdict | Diagnostic |
|---|---|---|---|
| d11 | `m:oMathPara` as a direct child of `w:body` | **REFUSED** | `decode-failed` |
| d12 | `w:dirty="true"` on the TOC begin `fldChar` | **accepted** | — |

**One of twelve.**

The eleven that pass are not marginal: **they are exactly the classes that make Word refuse the file.** d10 is the one that made Folio's output unopenable; d1–d5 are the ordering classes the schema catches; d9 is a corrupt embedded package. Plate's importer accepts all of them.

The one it catches, d11, is also the one that makes sense. `m:oMathPara` is a **block-level display equation** — an element whose parent is `w:body` rather than a paragraph — and Plate's schema has no node type for it. The decoder reaches a construct its own model cannot represent and fails with:

> `decode-failed`: DOCX content could not be decoded by the installed editor schema.

So Plate is not a validator. It has no schema of OOXML, no notion of element order, no awareness of `ST_LongHexNumber` or of a relationship pointing at garbage. It has a schema of **Plate's own node types**, and it notices when the document contains something that is not one of them. The failure is a *type* failure, not a *validity* failure — which is exactly why it catches one class and misses eleven.

The comparison with the other thirteen participants is the point. They either **corrupt** the malformed input (preserve the damage) or **discard** it (drop the part), and both outcomes are silent. Plate **declines**. Of the three possible behaviours on malformed input — repair, degrade, refuse — **only one of the fourteen implements the third**, and it is the one with the most stars.

### 4.4 What "repair" would even have looked like

For completeness, the three interventions that would have turned a REFUSED into an OPENED, none of which any editor performs:

1. **Re-emit the three fatal constructs in the shape Word accepts.** `w:delText` → `w:t` inside `w:moveFrom`; `m:oMathPara` → wrapped in a `w:p`; `w:dirty="true"` → `w:dirty="false"` or removed. Each is a one-attribute or one-element change, and each was proven to flip the verdict in the first report.
2. **Validate the output against the transitional schema before writing it.** This catches the five ordering classes and the malformed `ST_LongHexNumber`, and it is cheap: `build/validate_ooxml.py` does it in under a second for the whole corpus.
3. **Re-import the output and compare.** This catches the shape-5 failures — the ones where a writer emits markup it never validated, such as office-open's `<a:graphic>` with no `xmlns:a` — because the second pass is the one that fails to bind the prefix.

Not one editor does any of the three. The pass-through editors cannot, by construction: repairing a part requires modelling it, and modelling it is what pass-through refuses to do.

**Docxodus is the one participant that has a repair stage, and it repairs a different set of defects.** Its `PackageOpenRepair` funnel exists because *"Package defects Word opens but the Open XML SDK rejects"* would otherwise make the SDK refuse the file at `WordprocessingDocument.Open` (`Docxodus/DocxSession.cs:166`). It fixes four things: a relationship to a part with **no content type** or **no ZIP entry**, a part reached by a known relationship but **typed only as generic XML**, and **two relationships sharing an Id**. **None of those is one of this report's twelve defect classes** — which is exactly why the measurement above shows Docxodus repairing nothing.

That is the point worth keeping, and it generalises: **a repair stage is scoped to the consumer it unblocks.** A funnel that stops the SDK's own parser from refusing a file will not notice that `w:tcPr`'s children are in the wrong order, because the SDK is perfectly happy to read a badly ordered `w:tcPr` — it has no sequence check to make.

---

## 5. The causes, in seven shapes

Every malformation observed in this study — in the corpus and in the fourteen participants' outputs — falls into one of seven shapes. Each shape has a characteristic fix.

| # | Shape | Example | The fix |
|---|---|---|---|
| 1 | **Sequence written from a map, not from the schema's order** | Fidus: `w:footnotePr` after `w:compat`; Docxodus: the SDK re-prints `w:tcPr`'s children in the order it read them | emit children in the schema's sequence order |
| 2 | **The same relationship resolved twice, with two rules** | office-open: `sourceRidFor()` vs `sourceRelationshipFor()` | one resolver, used by both the body writer and the rels writer |
| 3 | **A model normalises a construct into a different one** | Folio: move source → deleted text | model the construct as it is; do not collapse a move into a deletion |
| 4 | **The model cannot represent the construct** | office-open's SmartArt fallback; canvas's chart drop | model it, or drop it *with a diagnostic* — but do not re-synthesize it wrongly |
| 5 | **Re-synthesis without a namespace registry** | office-open's second pass: `<a:graphic>` with no `xmlns:a` | one registry of namespaces for the whole document, used by every writer |
| 6 | **A required attribute omitted** | canvas: `<w:shd>` without `w:val` | validate against the schema before writing |
| 7 | **Format conversion with no home for the construct** | docs.plus: HTML has no sections, so `sectPr` lands first | keep a document model, not a document format |

Shapes 1, 3, 6 and 7 are **writer** bugs and are caught by a schema validator. Shape 2 is caught by a package-integrity check. Shape 5 is caught by re-importing the output. Shape 4 is a design decision that must at least be *reported*.

Shape 2 is the one that defeated everything else, and it is worth stating as a rule: **if two functions resolve the same identifier, they must resolve it the same way.** The asymmetry between `sourceRidFor()` and `sourceRelationshipFor()` is two lines of regex.

---

## 6. A detector catalogue

What to run, in increasing cost:

1. **Well-formedness.** Parse every XML part. Catches unbound prefixes and unclosed tags. Cheap; `xml.etree` is enough.
2. **Content-type coverage.** Every part must be covered by a `Default` (by extension) or an `Override`. Parse `[Content_Types].xml` — do **not** regex it: writers differ in attribute order, and a regex reports a false "no content type" for every part in a package that writes `ContentType` before `Extension`.
3. **Schema validation** against the ECMA-376 Transitional XSDs, with exactly two classes excluded: the markup-compatibility attributes and elements (`mc:Ignorable`, `mc:AlternateContent`, `mc:Choice`, `mc:Fallback`), and elements and attributes in Microsoft's extension namespaces (`w14`, `w15`, `w16du`, `w16cid`, `w16cex`) together with namespaces the transitional schema set does not model at all (`wps`, `wpg`, `wpc`, `wpi`) when they are demanded by a strict wildcard. Anything else the schema rejects is real.
4. **Relationship integrity.** Every `r:id`/`r:embed`/`r:link`/`r:dm`/`r:lo`/`r:qs`/`r:cs` used in a part must be declared in that part's `.rels`, and every internal relationship target must exist in the package. This is the check that catches office-open and Docen.
5. **Structural balance.** `w:fldChar` begin/separate/end must nest; a `w:moveFrom` must have a `w:moveTo` of the same name; `mc:Ignorable` must not name an undeclared prefix.
6. **Re-import.** Feed the output back into the editor that produced it. This is the only check that catches shape 5, and it is the only one that found office-open's second-pass defect.
7. **Microsoft Word.** The only verdict that decides whether the file is usable. Expensive, non-parallel, and it returns no diagnosis — but a chain of the six checks above reduces the need for it to the last one or two candidates.
8. **Decode against a model, not a schema.** A consumer that must build an internal model from the document will fail on constructs it cannot represent — Plate's `decode-failed: DOCX content could not be decoded by the installed editor schema.` is exactly this, and it is the **cheapest** detector in the catalogue, because the editor is already parsing and the failure is free. It is also the **weakest**: measured, it catches **one** of the twelve defect classes (§4.3), because it tests a schema of *node types* rather than a schema of *OOXML*. It is worth adding not because it is strong, but because it is the only detector that runs before the editor has done any damage.

9. **Per-defect isolation, for any detector you intend to trust.** A detector run against a corpus with all twelve defects at once tells you only that *something* is wrong. Each defect class was isolated into its own corpus (`build/make_single_defect.py`) and offered alone, which is how the table in §4.3 exists at all — and how the eleven-versus-one result was established rather than assumed.

---

## 7. Corrections to the first report

Four claims in the first report were imprecise. The code evidence behind them is better than the prose was, and the prose should say what the code says.

1. **docs.plus does not destroy *all* tracked changes at the mammoth stage.** mammoth's reader table gives `w:ins` and `w:moveTo` a reader (`mammoth/lib/docx/body-reader.js:447`, `:450`), so their **text** survives as ordinary visible HTML; `w:del` (`:729`) and `w:moveFrom` (`:732`) are in `ignoreElements` and are dropped silently. The revision *markup* — author, date, ids, the elements themselves — is destroyed. The accurate statement is that mammoth **flattens tracked changes to their accepted state**. `ignoreElements` is in fact a *warning-suppression* list, consulted at exactly one place (`body-reader.js:51-52`): an element in the map has no reader and is dropped **silently**; an element *not* in the map and without a reader is dropped **with a warning**. The first report's "destroys all tracked changes" was wrong about the text and right about the markup.
2. **Docen does not drop the tracked change inside `header2.xml` — it relocates it to `header3.xml`.** The first report's check was path-scoped; the content survives at a different path. (Docen also *adds* `header3/4/5` and `footer3`, which the part map records.)
3. **Fidus Writer's exporter does not fill `{@body}`.** The placeholder vocabulary is `{title}`, `{@<part-id>}` for rich-text and table parts, plus `{@bibliography}`, `{@copyright}` and `{@licenses}` (`src/exporter/template/tags.ts:18-101`).
4. **canvas-editor's exporter has no copy path at all.** It never opens the source package; `jszip` is imported only by the importer. So "0 of 35 parts pass through" is not a policy decision but a structural fact.
5. **"No editor diagnoses malformed input" was true of the first eleven and false of the twelfth.** Plate.js was added on 2026-10-30 and it **refuses** the broken corpus with a named diagnostic (`decode-failed`) rather than producing a damaged file. The precise statement is now: **no editor repairs malformed input; one of the fourteen detects it, and detects exactly one of the twelve defect classes.**
6. **"No editor repairs anything" is true of these twelve defect classes and false as a general statement.** Docxodus ships a repair funnel (`PackageOpenRepair`) that fixes four classes of package defect on the way in. It repairs none of the twelve here, because it is aimed at defects that would make the **Open XML SDK's own parser** refuse the file, not at defects that make **Microsoft Word** refuse it. The distinction is worth keeping for anyone reading the TL;DR literally: a repair stage is scoped to the consumer it unblocks, and Docxodus's is scoped to the SDK, not to Word.

---

## Reproducing this

```
build/make_broken_fixture.py         # the corpus with all twelve defects re-introduced
build/make_single_defect.py all      # one corpus per defect class, for the matrix in §4.3
build/run_broken.sh                  # every editor over the broken corpus
build/defect_survival.py             # which defect classes survive in each output
build/malformation_audit.py          # package integrity of the corpus and every output
build/validate_all_outputs.py        # XSD validation of every output
build/scan_word_hostile.py           # the three semantic defect classes, per output
build/word_open.py <file>            # ask Word to open one file
build/folio_deltext_test.py          # the decisive Folio repair experiment
build/officeopen_rels_test.py        # the decisive office-open / Docen repair experiment

# Docxodus, over the clean and the broken corpus:
cd harness/ui/docxodus && node roundtrip.mjs ../../fixture/complex.docx        ../../out/docxodus/complex.docx
cd harness/ui/docxodus && node roundtrip.mjs ../../fixture/complex_broken.docx ../../out_broken/docxodus/complex.docx
```

```
# Plate.js, headless, with the static presentation kit:
cd harness/plate
bun --preload ./aliases.ts roundtrip.ts ../../fixture/complex.docx ../../out/plate/complex.docx
bun --preload ./aliases.ts defect_probe.ts ../../out/defects     # the §4.3 matrix

# the two Word matrices, each ~25 s per file, run in the background:
python3 build/word_open.py out/<editor>/complex.docx ...          # clean corpus
python3 build/word_open.py out_broken/<editor>/complex.docx ...   # broken corpus
```

The two Word matrices in §4.1 and §4.2 are the only part of this report that cannot be re-run unattended in a few seconds: the harness restarts Word between files and waits for the AutoRecovery restore, so each verdict costs about twenty-five seconds, and a refusal has to be cleared with a hard kill before the next file can be tested.

**Plate.js needs a DOM realm and a Node runtime side by side.** Its importer requires `DOMParser`, `XMLSerializer`, `DOMException`, `HTMLElement` and `Node.TEXT_NODE` on `globalThis` (`import/lib/importDocx.ts:225-240`), which happy-dom supplies — but happy-dom's `GlobalRegistrator` also replaces the global `WritableStream` and `Blob` with non-spec-compliant versions, and `@zip.js/zip.js` then fails with `writable.getWriter is not a function`. `harness/plate/setup_dom.ts` registers happy-dom's DOM and restores Node's native `WritableStream`, `ReadableStream`, `TransformStream` and `Blob`. `harness/plate/aliases.ts` resolves the www app's `@/` alias so Plate's own static presentation kit can be used.

The code citations refer to the pinned commits listed in the first report, under `repos/`. None of those checkouts was modified.